BÁO CÁO BÀI TẬP GIỮA KỲ · KHÓA CKA 2026

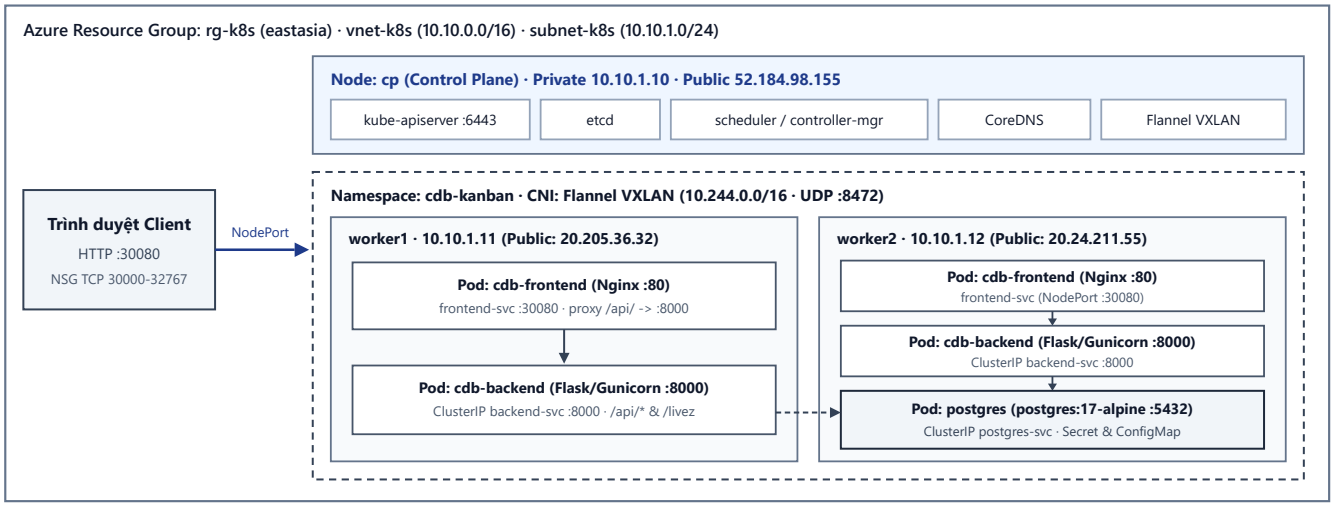
Triển khai ứng dụng Web CDB Kanban với Docker và Kubernetes (`kubeadm`) trên Microsoft Azure

Tài liệu trình bày hạ tầng Cloud trên Azure, kiến trúc ứng dụng Web 3 tầng độc lập, đóng gói Docker image theo phiên bản và kiểm thử vận hành cụm Kubernetes 3 node.

Repository GitHub	`https://github.com/quanq026/cdb-kanban-lab_k8s`
Docker Hub Images	`quanq026/cdb-kanban-frontend:v1`, `:v2` · `quanq026/cdb-kanban-backend:v1`, `:v2` (không dùng `:latest`)
Môi trường triển khai	Microsoft Azure (Resource Group `rg-k8s`, Region `eastasia`) · Kubernetes `v1.35.10` · `containerd://2.4.2` · Flannel VXLAN
Truy cập Demo (NodePort)	`http://20.205.36.32:30080` (`worker1`) · `http://20.24.211.55:30080` (`worker2`)

1. Sơ đồ kiến trúc Cluster & Hạ tầng Microsoft Azure

Hệ thống được dựng mới hoàn toàn trong Resource Group `rg-k8s` (region `eastasia`) trên mạng ảo riêng `vnet-k8s` (`10.10.0.0/16`, subnet `subnet-k8s` `10.10.1.0/24`). Lưu lượng bên ngoài được lọc qua Network Security Group `nsg-k8s` với các luật mở cổng TCP `22` (SSH), `80` (HTTP), `443` (HTTPS) và dải `30000-32767` (NodePort). Giao tiếp nội cụm giữa 3 node (`6443/TCP`, `10250/TCP`, Flannel VXLAN `8472/UDP`) thông qua luật mặc định `AllowVnetInBound`.



Hình 1: Kiến trúc cụm Kubernetes 3 node trên Azure và kết nối các Service trong Namespace `cdb-kanban`.

Node	Vai trò	Private IP	Public IP tĩnh	Thông số phần cứng & OS
`cp`	Control Plane (`apiserver`, `etcd`, `scheduler`, `controller-manager`)	`10.10.1.10`	`52.184.98.155`	`Standard_B2als_v2` (2 vCPU, 3.8 GiB RAM, `x86_64`), `StandardSSD_LRS` 30 GB, Ubuntu 24.04.5 LTS
`worker1`	Worker Node 1 (chạy Pod ứng dụng)	`10.10.1.11`	`20.205.36.32`	Cùng cấu hình như trên (kernel `7.0.0-1017-azure`)
`worker2`	Worker Node 2 (chạy Pod ứng dụng & PostgreSQL)	`10.10.1.12`	`20.24.211.55`	Cùng cấu hình như trên (kernel `7.0.0-1017-azure`)

2. Kiến trúc ứng dụng Web 3 tầng & Đóng gói Docker Image theo phiên bản

Ứng dụng quản lý công việc CDB Kanban trong repository `quanq026/cdb-kanban-lab_k8s` được xây dựng theo kiến trúc 3 tầng độc lập (Frontend – Backend API – Database), cho phép đóng gói, cấu hình và mở rộng riêng từng thành phần:

- **Frontend (`frontend/` — `nginx:1.27-alpine`, ~73.7 MB):** Phục vụ các trang tĩnh (`/`, `/login`, `/register`, CSS/JS). Tệp `nginx.conf` reverse proxy toàn bộ đường dẫn `/api/` tới `http://backend-svc:8000` (chuyển tiếp `Host`, `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`, `X-Forwarded-Host`) và mở endpoint `/healthz`. Script khởi động `99-pod-info.sh` sinh file `/frontend-info.json` ghi nhận version và hostname Pod để hiển thị trực tiếp lên giao diện.
- **Backend (`backend/` — `python:3.12-slim` + Flask + Gunicorn, ~217 MB):** Chỉ đảm nhiệm REST API (2 worker, cổng `8000`, chạy bằng user non-root `UID 10001`), dùng `ProxyFix` để nhận đúng thông tin từ Nginx. Bổ sung biến môi trường

`COOKIE_SECURE` (đặt `"false"` trong K8s để trình duyệt lưu session cookie khi truy cập qua HTTP NodePort), thêm endpoint `/livez` cho `livenessProbe` và mở rộng `/api/health` trả thêm `version` và tên `pod`.

- **Database (`db/schema.sql` — `postgres:17-alpine`):** Khởi tạo hai bảng `users`, `tasks`, chỉ mục `idx_tasks_user_status_position` và trigger `trg_tasks_updated_at` thông qua ConfigMap mount vào `/docker-entrypoint-initdb.d/01-schema.sql`.

Docker Hub Repository	Tag phiên bản	Base Image	Kích thước	Khác biệt giữa `v1` và `v2` (từ `ARG APP_VERSION`)
`quanq026/cdb-kanban-frontend`	`:v1`, `:v2`	`nginx:1.27-alpine`	~73.7 MB	Thay placeholder `__APP_VERSION__` trên HTML, sinh `/frontend-info.json` và đổi badge phiên bản trên CSS.
`quanq026/cdb-kanban-backend`	`:v1`, `:v2`	`python:3.12-slim`	~217 MB	Đặt `ENV APP_VERSION=v1|v2` để phản hồi qua JSON từ endpoint `/api/health`.

3. Các bước cài đặt Kubernetes (`kubeadm`) & Triển khai YAML

1. **Chuẩn bị hệ điều hành trên 3 node:** Tắt swap (`swapoff -a`, xóa cấu hình trong `/etc/fstab`), khai báo ánh xạ tên `cp` / `worker1` / `worker2` vào `/etc/hosts`, nạp module `overlay` và `br_netfilter`, bật `net.ipv4.ip_forward = 1` và `net.bridge.bridge-nf-call-iptables = 1`.
2. **Cấu hình Container Runtime & Cài đặt gói K8s:** Tạo lại `/etc/containerd/config.toml` bằng `containerd config default` trên cả 3 node để mở lại plugin CRI (`disabled_plugins = []`) và chuyển `SystemdCgroup = true` cho `containerd v2.4.2`. Cài đặt và khóa phiên bản (`apt-mark hold`) cho `kubelet`, `kubeadm`, `kubectl` bản `v1.35.10`.
3. **Khởi tạo Control Plane, cài CNI & Join Worker:** Chạy `sudo kubeadm init --apiserver-advertise-address=10.10.1.10 --pod-network-cidr=10.244.0.0/16 --node-name=cp`, cấp cấu hình `~/.kube/config`, triển khai **Flannel VXLAN** (`kube-flannel.yml`) và chạy lệnh `kubeadm join` trên `worker1`, `worker2`.
4. **Triển khai các tệp YAML trong `k8s/` (`kubectl apply -f ./k8s/`):**

Tệp YAML	Tài nguyên Kubernetes	Cấu hình chính
`00-namespace.yaml`	`Namespace/cdb-kanban`	Cô lập toàn bộ tài nguyên của hệ thống với nhãn `app.kubernetes.io/part-of: cdb-kanban`.
`01-postgres.yaml`	`Secret/cdb-secrets`, `ConfigMap/postgres-initdb`, `Deployment/postgres`, `Service/postgres-svc`	`postgres:17-alpine`, 1 replica (`strategy: Recreate`), nạp `01-schema.sql`, dữ liệu dùng `emptyDir`, probe `pg_isready`, resource `50m/128Mi` – `500m/512Mi`, mở `ClusterIP :5432`.
`02-backend.yaml`	`ConfigMap/backend-config`, `Deployment/cdb-backend`, `Service/backend-svc`	`DB_HOST="postgres-svc"`, `COOKIE_SECURE="false"`; 2 replica ảnh `:v1`; `RollingUpdate` (`maxSurge: 1, maxUnavailable: 0`); `topologySpreadConstraints` (`ScheduleAnyway`); `tolerations: 30s`; probe `/api/health` & `/livez`; `ClusterIP :8000`.
`03-frontend.yaml`	`Deployment/cdb-frontend`, `Service/frontend-svc`	2 replica ảnh `:v1`; giữ cùng cấu hình HA, RollingUpdate & tolerations 30s như Backend; probe `/healthz`; công bố qua `NodePort 80:30080`.

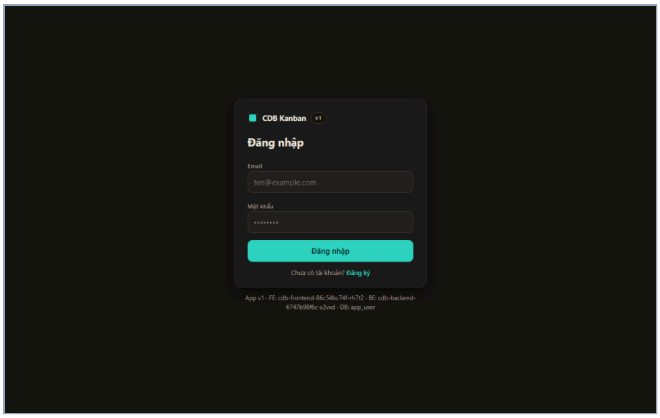
4. Kết quả triển khai & Kiểm thử vận hành thực tế

4.1. Trạng thái Cluster 3 Node & Triển khai phiên bản `v1`

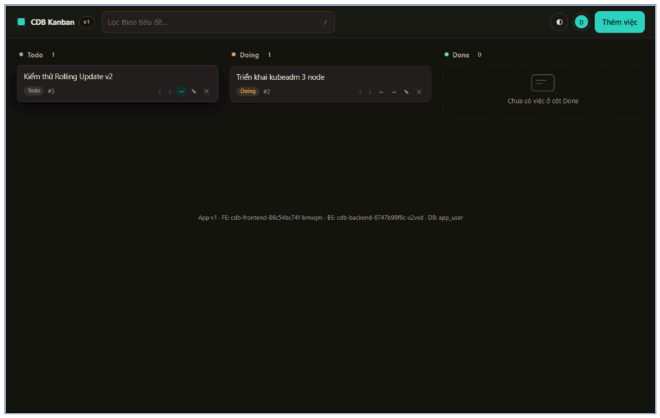
```
$ kubectl get nodes -o wide
NAME      STATUS   ROLES           AGE   VERSION    INTERNAL-IP   OS-IMAGE             CONTAINER-RUNTIME
cp        Ready    control-plane   50m   v1.35.10   10.10.1.10    Ubuntu 24.04.5 LTS   containerd://2.4.2
worker1   Ready    <none>          50m   v1.35.10   10.10.1.11    Ubuntu 24.04.5 LTS   containerd://2.4.2
worker2   Ready    <none>          50m   v1.35.10   10.10.1.12    Ubuntu 24.04.5 LTS   containerd://2.4.2

$ kubectl get pods,svc -n cdb-kanban -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
pod/cdb-backend-6747b98f6c-bjctm    1/1     Running   0          60m   10.244.1.9    worker1
pod/cdb-backend-6747b98f6c-x2vxd    1/1     Running   0          59m   10.244.2.13   worker2
pod/cdb-frontend-86c54bc74f-bmxqm   1/1     Running   0          60m   10.244.1.10   worker1
pod/cdb-frontend-86c54bc74f-rh7t2   1/1     Running   0          59m   10.244.2.12   worker2
pod/postgres-b457ffcd6-9srxc        1/1     Running   0          64m   10.244.2.2    worker2

NAME                   TYPE        CLUSTER-IP       PORT(S)        SELECTOR
service/backend-svc    ClusterIP   10.99.248.222    8000/TCP       app=cdb-backend
service/frontend-svc   NodePort    10.106.144.93    80:30080/TCP   app=cdb-frontend
service/postgres-svc   ClusterIP   10.107.241.226   5432/TCP       app=postgres
```



Hình 2: Truy cập thực tế `http://20.24.211.55:30080/login` hiển thị phiên bản `v1` và tên Pod FE/BE



Hình 3: Tạo task và chuyển cột `Todo -> Doing` trên `http://20.205.36.32:30080` (lưu xuống PostgreSQL)

4.2. Kiểm thử Scale 4 Replicas, Rolling Update (`v1 → v2`) & Rollback

```
$ kubectl scale deployment cdb-frontend -n cdb-kanban --replicas=4
deployment.apps/cdb-frontend scaled
$ kubectl get pods -n cdb-kanban -l app=cdb-frontend -o wide
NAME                            READY   STATUS    RESTARTS   AGE   IP           NODE
cdb-frontend-7c4fb64f7c-d4vll   1/1     Running   0          8s    10.244.1.4   worker1
cdb-frontend-7c4fb64f7c-lzq8k   1/1     Running   0          72s   10.244.1.3   worker1
cdb-frontend-7c4fb64f7c-s4xhv   1/1     Running   0          8s    10.244.2.5   worker2
cdb-frontend-7c4fb64f7c-xxmb6   1/1     Running   0          72s   10.244.2.4   worker2

$ kubectl set image deployment/cdb-backend backend=quanq026/cdb-kanban-backend:v2 -n cdb-kanban
$ kubectl set image deployment/cdb-frontend frontend=quanq026/cdb-kanban-frontend:v2 -n cdb-kanban
$ curl -s http://20.205.36.32:30080/api/health
{"db_user":"app_user","ok":true,"pod":"cdb-backend-5dfcf77654-2bwk7","status":"ok","version":"v2"}

$ kubectl rollout undo deployment/cdb-backend -n cdb-kanban
$ kubectl rollout undo deployment/cdb-frontend -n cdb-kanban
$ curl -s http://20.205.36.32:30080/api/health
{"db_user":"app_user","ok":true,"pod":"cdb-backend-784bfd57c5-tdqtp","status":"ok","version":"v1"}
```

Deployment `cdb-frontend` mở rộng lên 4 replica chia đều 2 Pod mỗi worker trước khi thu gọn về 2. Khi nâng cấp lên `:v2`, nhờ `maxUnavailable: 0` và `readinessProbe`, Kubernetes đảm bảo Pod mới sẵn sàng trước khi ngắt Pod cũ; toàn bộ dữ liệu user/task ở PostgreSQL được giữ nguyên và `kubectl rollout undo` khôi phục về `version: "v1"` không gián đoạn dịch vụ.

4.3. Thử nghiệm tắt 1 Worker Node (`worker1`) & Khôi phục

```
$ az vm deallocate -g rg-k8s -n worker1
$ kubectl get nodes
NAME      STATUS     ROLES           AGE     VERSION    INTERNAL-IP
cp        Ready      control-plane   9m28s   v1.35.10   10.10.1.10
worker1   NotReady   <none>          9m17s   v1.35.10   10.10.1.11
worker2   Ready      <none>          9m17s   v1.35.10   10.10.1.12

$ kubectl get pods -n cdb-kanban -o wide
NAME                            READY   STATUS        RESTARTS   AGE     IP             NODE
cdb-backend-784bfd57c5-cflzp    1/1     Running       0          11s     10.244.2.10    worker2
cdb-backend-784bfd57c5-l79gs    1/1     Running       0          11s     10.244.2.11    worker2
cdb-backend-784bfd57c5-nwx2l    1/1     Terminating   0          99s     10.244.1.8     worker1
cdb-frontend-7c4fb64f7c-5x5hl   1/1     Running       0          106s    10.244.2.8     worker2
cdb-frontend-7c4fb64f7c-wlr7z   1/1     Running       0          99s     10.244.2.9     worker2
postgres-b457ffcd6-9srxc        1/1     Running       0          3m37s   10.244.2.2     worker2
```

Trong trạng thái cụm trên, Pod `postgres` đang chạy tại `worker2` nên nhóm tiến hành tắt nóng node `worker1` qua `az vm deallocate`. Node `worker1` chuyển sang `NotReady`; nhờ `tolerationSeconds: 30` và `whenUnsatisfiable: ScheduleAnyway`, Kubernetes nhanh chóng khởi tạo lại Pod bị mất sang `worker2` để đưa cả Backend và Frontend về lại `2/2 Running`. Truy cập web qua IP node sống `http://20.24.211.55:30080` vẫn đăng nhập và thao tác Kanban bình thường. Sau khi bật lại `worker1` bằng `az vm start`, node tự động quay lại trạng thái `Ready`. (Ghi chú: Do lab sử dụng `emptyDir` cho PostgreSQL nên thao tác tắt node được thực hiện trên `worker1` không chứa DB.)

5. Các khó khăn gặp phải & Giải pháp xử lý

1. Giới hạn SKU máy ảo & Quota Public IP trên Azure: Vùng `japaneast` chặn cấp phát dòng máy ảo B-series `x86_64` và subscription giới hạn tối đa 3 Public IPv4 Standard mỗi region. *Giải pháp:* Chuyển sang region `eastasia` sử dụng SKU `Standard_B2als_v2` (tổng 6 vCPU, 3 Static IP) vừa đủ cho 1 Control Plane và 2 Worker.

2. Gói `containerd.io` mặc định tắt CRI & sai Cgroup driver: File `/etc/containerd/config.toml` cài sẵn từ repo Docker có `disabled_plugins = ["cri"]` và `SystemdCgroup = false` làm `kubeadm init` lỗi. *Giải pháp:* Tái tạo file cấu hình bằng `containerd config default` và đặt `SystemdCgroup = true` trên cả 3 node.

3. Mạng ảo Azure SDN chặn gói tin IP-in-IP: Đường hầm IP-in-IP giữa các VM thường bị drop trong Azure VNet. *Giải pháp:* Cài đặt CNI **Flannel VXLAN** (`10.244.0.0/16`) đóng gói qua UDP cổng `8472` để đảm bảo Pod giao tiếp thông suốt xuyên node.

4. Trình duyệt không lưu Session Cookie khi dùng HTTP NodePort: Cấu hình mặc định của cookie phiên bật cờ bảo mật `SESSION_COOKIE_SECURE=True` (yêu cầu HTTPS), khiến trình duyệt từ chối lưu cookie khi truy cập qua HTTP thuần. *Giải pháp:* Cho phép điều chỉnh qua biến môi trường `COOKIE_SECURE="false"` trong `ConfigMap/backend-config` cho endpoint `http://<node-ip>:30080`.

5. Rải đều Pod nhưng vẫn tự phục hồi nhanh khi mất Node: Nếu dùng `DoNotSchedule`, khi mất 1 worker sẽ có Pod bị kẹt `Pending`, đồng thời mặc định K8s đợi 300s mới loại Pod trên node lỗi. *Giải pháp:* Sử dụng `whenUnsatisfiable: ScheduleAnyway` trong `topologySpreadConstraints` kèm `tolerationSeconds: 30`.

6. Cấu hình SSH bằng mật khẩu trên Ubuntu 24.04 Cloud Image: Các tệp `50-cloud-init.conf` và `60-cloudimg-settings.conf` ép `PasswordAuthentication no` theo thứ tự ưu tiên đọc trước của OpenSSH. *Giải pháp:* Tạo tệp `00-password-auth.conf` đứng đầu thứ tự bảng chữ cái và cập nhật đồng bộ cả 3 máy ảo.

6. Tổng kết & Đối chiếu tiêu chí hoàn thành

Hạng mục	Kết quả triển khai & Kiểm chứng
Ứng dụng Web & Docker	Thiết kế độc lập `frontend/` & `backend/`, build và push 4 image tag `:v1`, `:v2` lên Docker Hub (không dùng `:latest`); 19/19 unit test và `ruff check` đều đạt.
Cluster K8s 3 Node	3 VM Azure (`cp`, `worker1`, `worker2`) cài `kubeadm v1.35.10` + Flannel VXLAN, hiển thị 3/3 node `Ready`.
Manifests YAML	Đầy đủ `Namespace`, `Secret`, `ConfigMap`, `Deployment` (liveness/readiness probe, HA), `ClusterIP` & `NodePort :30080`.
Vận hành & Giao diện	Đã kiểm chứng `scale` 4 replicas, rolling update `v1 → v2`, `rollout undo`, failover khi tắt `worker1` và truy cập trực tiếp qua trình duyệt.

Hướng phát triển mở rộng: Khi đưa hệ thống vào vận hành thực tế (production), có thể bổ sung thêm Ingress Controller kết hợp chứng chỉ TLS để bật lại `COOKIE_SECURE="true"` thay cho HTTP NodePort, đồng thời chuyển Deployment PostgreSQL sang `StatefulSet` dùng `PersistentVolumeClaim` (Azure Disk CSI Driver) nhằm bảo toàn dữ liệu vĩnh viễn khi nút cơ sở dữ liệu gặp sự cố.